Highly Scalable and Reliable News Feed System Design

Generated on 2025-08-10 02:44 UTC

Highly Scalable and Reliable News Feed System Design
Generated on 2025-08-10 02:44 UTC
One Pager Summary
Goal: Deliver a real time, personalized news feed that users can read and interact with at web scale while meeting strong reliability, latency, and privacy standards.
Users: End users reading and posting content, creators, moderators, tenant administrators in a multi tenant SaaS offering for brands or communities.
Core Value: Fast home feed with relevant ranking, instant interactions like like, comment, repost, follow, and robust moderation.
North Star Metrics: p95 feed fetch under 150 milliseconds from edge cache, p95 post publish to follower visibility under 2 seconds, availability 99.95 percent or better, data durability eleven nines for events and media references.
Scope: Create and consume posts, like, comment, repost, follow and unfollow, notifications, search and discovery, moderation and safety, analytics.

Functional Requirements

Create, edit, and delete posts with text, links, images, and video references.
Home feed with personalization and relevance ranking. Latest feed showing reverse chronological items.
Interactions: like, comment, repost, share, bookmark. Edit and delete with audit history.
Follow graph: follow and unfollow with privacy controls for public or private accounts.
Notifications for mentions, replies, follows, and interaction summaries.
Moderation: report content, automated policy enforcement, human review, and block or mute.
Search and discovery: full text search with filters and trending topics.
Multi tenancy: separate spaces for different organizations with brand level configuration.
Observability: per tenant dashboards for usage, latency, and health.
Admin tooling: content takedown, user restrictions, and legal hold exports.
Non
Availability target 99.95 percent platform wide and 99.99 percent for read heavy endpoints.
Latency: p95 feed read under 150 milliseconds from edge cache, p95 write to visibility under 2 seconds, p95 interactions write under 120 milliseconds.
Consistency: Time ordered correctness within a user timeline. Eventual consistency across regions within 2 to 5 seconds. Read your writes for author on own content.

Durability: Eleven nines for append only activity events and six nines for operational data across multiple availability zones.
Scalability: Serve 1 billion daily feed reads and 100 million daily write events globally with bursts during peak news moments.
Privacy and compliance: SOC 2 type 2 readiness, GDPR, and CCPA aligned with data minimization and deletion workflows.
Cost efficiency: Majority of reads from cache and pre computed timelines; media stored in object storage with life cycle policies.

Architecture Overview

Client: Web and mobile apps using a typed SDK. The SDK uses a single script tag on web, async loaded, and establishes a session with short lived tokens.
Edge: CDN and edge compute for authentication checks, cache, and lightweight aggregation. Push notifications fan out via platform services.
Gateway: API Gateway with WAF and DDoS protection. REST and GraphQL interfaces. Request routing by service.
Core Services

Identity and Sessions: OAuth and OIDC for app users and tenant admins. Device binding and session rotation.
Social Graph Service: manages follow edges and privacy policy checks.
Content Service: manages posts, media references, edit history, and visibility.
Activity Events Service: accepts interactions like like and comment and repost as immutable events.
Feed Orchestrator: coordinates write fan out and read assembly between fan out on write and fan out on read paths.
Ranking Service: online features and model inference for personalization with feature store support.
Timeline Store: materialized timelines per user and per list with hot segments cached.
Search Service: indexes posts and author signals for discovery and trends.
Notification Service: generates alerts and digest notifications.
Moderation Service: automated policy checks, queues for human review, and enforcement.
Media Service: upload, transcode, thumbnail, and CDN distribution.
Analytics and ETL: exports events into data lake for offline training and reports.
Data Flow
Post Publish Path
Client posts content to Content Service which stores metadata in an operational store and writes a post created event to the message log. The Feed Orchestrator consumes the event. For fan out on write, it

computes the follower set using the Social Graph and enqueues timeline append commands. For large fan out or celebrity accounts, it records a pointer only and relies on fan out on read. The Timeline Writer updates user timeline shards and invalidates edge cache. Ranking features are updated and a notification is created.
Feed Read Path
Client requests the home feed. Edge cache attempts to serve from cached segments. On miss, the Feed Reader merges pre materialized timeline segments with on demand expansions for heavy hitters. The Ranking Service scores candidates using recency, author affinity, quality, and diversity constraints. Results are cached at edge with a short ttl and keyed by user id and segment position.
Interaction Path
Likes and comments are written to the Activity Events Service, persisted in the log, counted in counters with atomic increments, and reflected in cached objects. Read your writes is achieved by merging client side optimistic updates with server confirmations.
Technology Stack
Client: TypeScript SDK for web and mobile, Web Push and native push.
Edge: CDN with edge compute and key value cache.
Gateway: Envoy or API Gateway and WAF.
Messaging: Kafka or Kinesis partitioned by user id and post id as appropriate.
Operational Data: Postgres for strong relational needs with read replicas or DynamoDB for elastic scale. Use row level security for multi tenancy.
Timeline Store: Redis Cluster or RocksDB backed stream store for hot segments, plus Cassandra or DynamoDB for long tail time series.
Search: OpenSearch or Elastic managed service.
Media: Object storage with multipart upload and media processing pipeline.
Feature Store and Ranking: Online feature store on Redis or DynamoDB and offline store in data lake. Real time inference via model serving platform.
Observability: OpenTelemetry, Prometheus, Grafana, centralized logs.
CI CD and IaC: Git based pipelines, Terraform, container registry, progressive delivery with canaries and feature flags.
Data Model Sketch
Tenant: id, name, plan, quotas, region, settings, status, createdAt.
User: id, tenantId, handle, profile, visibility, metrics, createdAt.
FollowEdge: followerId, followeeId, tenantId, status, createdAt.
Post: id, tenantId, authorId, text, media list, visibility, replyTo, createdAt, editedAt, version.
TimelineItem: ownerUserId, postId, score, createdAt, shardId, source fan out write or read.

ActivityEvent: id, type like or comment or repost, userId, postId, tenantId, ts, meta, riskScore.
Notification: id, userId, type, entityId, status, ts.
ModerationCase: id, entityId, type, status, reviewerId, actions, ts.
Scaling and

Storage Strategy

Follow Graph
Store edges in a compressed adjacency list. Maintain counts and sample sets for fast follower retrieval. For celebrity fan out, maintain follower lists in shards with pagination to support background fan out.
Timelines
Hybrid approach. Fan out on write for normal accounts to pre compute user timelines. Fan out on read for heavy hitter posts using pointers and expanding during read. Maintain per user timeline shards by time range and store recent N items in Redis.
Counters
Use Redis or DynamoDB atomic counters for likes and comments with periodic reconciliation. Store per post aggregated counters and per user activity summaries.
Search and Trends
Ingest post and interaction events into search index with filters for tenant and visibility. Compute trending topics using rolling windows and z scores.
Cold Storage
Move posts older than threshold to cheaper storage while keeping metadata hot for search and direct access.
Caching
Use write through cache for posts and profiles and edge cache for timeline segments with short ttl and soft validation using ETags.
Latency Budgets and Throughput
Read path p95 target 150 milliseconds from edge cache for first page. Origin miss adds 60 to 100 milliseconds intra region.
Publish to visibility p95 target under 2 seconds including message propagation, timeline writes, and cache invalidation.
Interactions p95 write under 120 milliseconds with immediate optimistic confirmation.
Throughput example: 20 thousand posts per second peak, 2 million feed reads per second globally, 10 million likes per minute peak during events. Autoscaling based on consumer lag and cache miss rates.
SLA and SLOs

Platform monthly availability 99.95 percent. Feed reads 99.99 percent availability served by edge cache. Error budget of 0.5 percent.
RPO zero for event log and media references. RTO per region under 15 minutes with automatic failover to warm standby or active active topology.
Ranking and Relevance
Online scoring model combining recency, user affinity, content quality, and diversity constraints. Features include follow distance, past engagement, freshness decay, media type, and predicted dwell time.
Exploration versus exploitation via epsilon strategy or Thompson sampling within safe bounds. Guardrails to avoid echo chambers by injecting diverse content at fixed intervals. Ranking outputs explainability fields for audits.
Abuse resistant inputs using debiased counters and bot filtering to avoid gaming.
Moderation and Safety
Pre publish checks with policy rules and lightweight ML classifiers for toxicity and spam. Post publish monitoring with velocity and similarity detectors. Shadow bans and rate limits for suspected bots.
User controls to block, mute, and restrict replies. Per tenant policy packs with jurisdiction aware rules. Audit trail for all enforcement actions.
Security Privacy and Compliance
PII minimization and pseudonymous identifiers by default. Encryption in transit and at rest with KMS and automatic key rotation. Secret management with dedicated KMS and short lived credentials.
Data retention defaults with configurable policies per tenant. Data subject request workflows and re keying for

right to be forgotten.
SOC 2 controls for change management, access, and monitoring. GDPR and CCPA program including DPA, SCCs, and data residency options in EU. Vendor risk and annual pen tests.
Multi Tenancy
Logical isolation by tenant id enforced at application and database layers with row level security where supported. Per tenant rate limits, quotas, and custom domains. Dedicated VPC and regional isolation available for enterprise tier. Per tenant encryption keys and audit logs.
CI CD and Deployment
Trunk based development with short lived branches. Automated tests and security scans in CI. Infrastructure as code with Terraform and environment workspaces. Progressive delivery with canary releases, shadow reads for new ranking versions, and automatic rollback on SLO regression. Feature flags for phased rollouts.

Observability and Operations

Tracing across gateway, services, and data stores using OpenTelemetry. Metrics include QPS, p95 and p99 latencies, cache hit rates, consumer lag, ranking time, and error budgets. Structured logs with redaction. Alerting on SLO burn, cache miss spikes, consumer lag, slow ranking, and moderation backlog. Runbooks linked from alerts. Dashboards per tenant and region.
Chaos and resilience testing with queue pauses, cache flush, single shard failures, and regional outages. Quarterly game days.

Capacity Planning and Cost Estimates

Reference scale per region. Timeline: Redis cluster with twelve shards sized for two million keys per shard. Kafka three to five brokers with storage retention of three days. Postgres or DynamoDB write capacity for 50 thousand writes per second sustained. Search three data nodes and two masters for high availability. Object storage egress optimized via CDN.
Estimated monthly cost in tens to low hundreds of thousands of dollars depending on region count, traffic mix, and media usage. Cost levers include edge cache ttl, cold storage tiering, and delayed indexing for low value content.

Testing Strategy and Failure Drills

Unit tests for validators, ranking functions, and privacy checks. Property based tests for ordering and tie breaking. Contract tests for GraphQL and REST. Load and soak tests with spike scenarios for major events. Replay tests using production like logs in a staging environment.
Failure drills cover message log outage, cache failover, ranking service timeouts, search cluster degradation, and regional failover. Verify bounded latency increases, zero data loss, and automatic recovery.

Phased Build Plan

MVP four to six weeks. Create and read posts, simple follow graph, reverse chronological feed, likes, notifications, search basics, moderation reports, single region with edge caching.
Phase 2 two months. Ranking service, hybrid fan out, media pipeline, advanced moderation, analytics exports, multi AZ, and canary deploys.
Phase 3 three to six months. Active active multi region, personalized discovery, data residency, enterprise tenant isolation, ML model lifecycle with offline training and online validation.
Key Design Choices Trade Offs and Risks
Hybrid fan out balances write and read costs and limits blast radius for celebrity accounts, but increases read time complexity. Mitigation is smart caching and pre expansion for expected hot posts.
Ranking model introduces opacity and fairness risk. Mitigation is explainability fields, offline bias checks, and per tenant policy levers.

Search and timeline duplication raises consistency work. Mitigation is event driven updates and idempotent writers.
Abuse and spam adapt quickly. Mitigation is layered defenses, feedback loops, and model iteration.